

Operating Systems: Deadlocks

Introduction to Deadlocks

A deadlock is a situation in computing where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Consider an example when two trains are coming toward each other on the same track and there is only one track, neither of the trains can move once they are in front of each other. A similar situation occurs in operating systems when there are two or more processes that hold some resources and wait for resources from other(s).

Four Necessary Conditions for Deadlock

1. Mutual Exclusion: At least one resource must be held in a non-shareable mode. If another process requests that resource, the requesting process must be delayed until the resource has been released.
2. Hold and Wait: A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
3. No Preemption: Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
4. Circular Wait: A set of waiting processes must exist such that P₀ is waiting for a resource held by P₁, P₁ is waiting for a resource held by P₂... and P_n is waiting for a resource held by P₀.

Deadlock Handling

There are three ways to handle deadlock:

- Deadlock prevention or avoidance (ensure the system will never enter a deadlock state).
- Deadlock detection and recovery (let the system enter a deadlock state, detect it, and recover).

Operating Systems: Deadlocks

- Ignore the problem and pretend that deadlocks never occur in the system (used by most operating systems, including UNIX and Windows).